

SarkariSewa Portal — Full

Project Roadmap

Last updated: 18 July 2026. Upload this file back into a new chat with Claude to continue exactly where this planning session left off.

Where things stand right now

- **Module 1–7:** Core site built (80 services, 6 categories, bilingual EN/HI, dark/light theme, support pages, legal pages). Module 7's broken links (about/contact/faq/sitemap pages, dead `relatedServices` references) were audited and fixed.
 - **Module 8:** DONE. `search.html` (was missing — homepage search was 404ing), `robots.txt`, `sitemap.xml` (auto-generated via `generate-sitemap.py`), dynamic Schema.org JSON-LD + meta tags on category/service pages, header search icon.
 - **Module 9 and 10:** NOT started yet.
-

Module 9 — Blog System (not started)

- `blog/index.html` — post list (title, date, excerpt, category tag)
- `blog/post.html?slug=...` — single post template, JSON or Markdown-driven
- `data/blog-posts.json` (or individual `.md` files) — content source
- Bilingual posts (EN/HL), reuse existing header/footer/theme/i18n pattern
- Add blog URLs to `generate-sitemap.py` output
- Optional: "Latest from Blog" section on homepage; "Related posts" on post pages

Module 10 — Final QA & Launch Prep (not started)

- Full automated broken-link/asset audit (like the one done manually for Module 7)
- Mobile responsiveness pass, 320px up
- Accessibility pass — focus states, alt text, aria labels, contrast
- Lighthouse/SEO audit — speed, meta completeness, structured data validation
- 404 page

- Favicon/manifest check
- Final content accuracy review (helpline numbers, fees, links)

Decision made: publish the site right after Module 10, before starting anything else. Reasons:

- Domain age helps SEO — earlier live date = more trust built by the time later modules ship
- Real traffic/search data from the live site will inform later feature design (which districts/services get demand)
- The eventual CSC pitch to owners ("get leads from our traffic") only works if there's real traffic already
- Bugs surface faster with real users on real devices
- AdSense approval takes time and favors an established site
- Module 10's robots.txt/sitemap.xml should be submitted to Google Search Console right after publish to kick off indexing early

Module 11 — Design & Branding

Fixes

(Found via actual code inspection during Module 8, not guesses — see details below.)

Bugs (functional, not cosmetic)

1. Dark mode: hero search bar is invisible. `.hero`

and `.search-form` both use `var(--color-surface)` as background in dark mode, with only a low-contrast border between them — the search box visually disappears into the hero section.

2. Dark mode: search button text is invisible.

`.search-form button` uses `background: var(--color-primary) + color: #fff`. In dark mode, `--color-primary` flips to a near-white color (`#E8EDF3`) because it's reused as the heading/text color — so it becomes white text on near-white background. Same root-cause bug repeats in **5 places total**:

- `.search-form button` (style.css)
- `.btn-primary` (style.css)
- `.admin-card button` (style.css)
- RTI guide step-number badge (module7.css)
- `.chip--active` filter chip on search.html (module8.css)
- **Root cause:** `--color-primary` is overloaded — used both as "brand navy" for button backgrounds AND as "heading text color" that intentionally flips light/dark with theme. **Fix:** introduce a separate `--color-brand` (fixed navy, doesn't flip with theme) for anything that needs a solid-color button background, and

leave `--color-primary` for text/headings only.

3. "नवीनतम अपडेट" (Latest Updates) section is **mislabeled**. It just shows the last 6 entries of `services.json` array order — there's no `dateAdded` field in the data at all, so nothing is actually "latest." Fix:

- Add a `dateAdded` field to each service in `services.json` (needed for genuine "latest" sorting, and useful for future CSC/blog "recently added" sections too)
- Increase shown count from 6 to ~10–12 (not 20 — a long homepage hurts more than it helps; the real dwell-time lever is relevance/curation, not raw count)
- Add a clear "View all 80+ services →" CTA linking to `search.html`
- Consider curating so at least one service from each of the 6 categories appears

Missing assets

4. **Zero images anywhere on the site.** No favicon, no logo image (current logo is a text/CSS "S" letter-mark, fine for perf, but a favicon is still needed), no illustrations, no `og:image` for social share previews.

Monetization/CSC readiness

5. No ad space reserved anywhere in the layout.

When AdSense goes in, un-reserved ad slots cause layout shift (hurts Core Web Vitals → hurts ranking). Fix: add fixed-size placeholder containers in planned ad positions now, even empty.

6. No trust-building elements on homepage (e.g. a stats strip: "80+ Services · 6 Categories · Bilingual").

Module 12 — Per-Service Content Enhancements

Ideas gathered from studying govtschemes.in, myscheme.gov.in, and sarkariyोजना.com (large Indian govt-info sites) — see notes below each.

1. **"Common Issues / Troubleshooting" section per service** — e.g. "payment not received," "application rejected." Pure content (`commonIssues` array in `services.json` , rendered on `service.html`), no backend needed. Matches real search intent people type into Google.
2. **Structured "Summary box" at the top of each service page** — scheme/service name, launch info, benefit, who can apply, official portal, all in one scannable table.

3. **Tag-based "Matching Services"** — beyond category, add lightweight tags like person-type (Women, Senior Citizen, Student) and benefit-type (Financial Assistance, Health, Education) so related services cross-link more precisely than the current category-only relation. Improves internal linking for SEO too.

Module 13 — Discovery Features

1. **Downloadable master PDF** of all services/categories — low effort, high perceived value, generate once as a static file (regenerate when `services.json` changes, similar to `generate-sitemap.py`).
2. **"Find Services For You" eligibility wizard** — inspired by the official `myScheme.gov.in` platform's flow: user answers a few quick questions (state, category of interest, etc.) and gets a filtered list back. Fully client-side, no backend needed — built on the same `services.json` / `categories.json` data already in use.

Module 14 — Backend Foundation (Supabase)

- Set up the Supabase project and core schema

- This is a **shared foundation** – Modules 15–16 (comments, subscribe) and the later CSC modules (17–21) will all build on this same backend, so it only needs to be set up once.

Module 15 – Comments / Q&A on Service Pages

- Lightweight comment thread under each service page (Supabase-backed)
- Why: on govtschemes.in, scheme pages have 100s of comments – real engagement, a freshness signal for Google, and free long-tail keyword capture (people type their actual problem as a comment)

Module 16 – Email / WhatsApp Subscribe

- "Get updates about this service" – simple email capture (saved to Supabase) + WhatsApp channel link
 - Builds a return-visitor/lead channel ahead of the CSC modules, where this pattern gets reused for CSC-specific updates too
-

Module 17 onward – CSC Centre Listings Feature

Critical architecture note

Everything built in Modules 1–13 is a **fully static site** – plain HTML/JS/JSON, no backend, no database, no login system (Module 14 changes this, but only for comments/subscribe). Claim/edit/ownership/6-month-tracking flows for CSC need the same Supabase backend, extended with new tables – nothing about Modules 1–16 needs to change for this, the CSC section is purely additive (new `csc/` folder, new nav link).

Backend decision: Supabase (confirmed choice, already set up in Module 14)

Why, vs. Firebase:

- CSC data is inherently relational – CSC listing ↔ Owner account ↔ Claim request ↔ Leads, all linked with foreign keys, needs joins. Supabase is Postgres (proper SQL, relations); Firebase is Firestore (NoSQL documents) – better fit for
- Supabase pricing is flat/predictable (\$25/mo Pro tier covers up to 100K monthly active users);

Firebase bills per-operation and can spike unpredictably as traffic grows.

- Row-Level Security in Postgres gives cleaner per-owner data access control for the claim/edit flow.
- No vendor lock-in – standard Postgres, exportable any time.

Module 17 – CSC Public Directory

- Data sourcing: seed initial CSC data from an official government CSC locator dataset (e.g. data.gov.in) or manual entry – needs deciding when we get here
- `csc/index.html` – browse by **state** → **district**, reusing the existing `support/state-wise-services.html` browse pattern
- `csc/profile.html?id=...` – individual CSC page: name, address, services offered, hours, contact
- `LocalBusiness` Schema.org markup on every CSC profile page (same dynamic-injection pattern used for `GovernmentService` / `ItemList` in Module 8)
- Add CSC URLs to `generate-sitemap.py`

Audience-facing UX (confirmed requirements):

- Clean state/district-wise browsing, listings clearly visible

- **Verified/claimed CSCs only** get: "Request Service" button (lead capture form), "Call Now" (`tel:` link), **WhatsApp button** (`wa.me/` deep link)
- Unverified/unclaimed listings show basic info only
 - no contact/request buttons – plus a "Is this your CSC? Claim it" prompt

SEO strategy for CSC pages (programmatic local SEO – same model as JustDial/Sulekha):

- Real upside: long-tail local search traffic ("CSC centre [district]", "Aadhaar seva kendra near [pincode]"), state/district hub pages drive internal linking/crawl discovery
- Real risk: **thin/duplicate content** – hundreds of near-identical auto-filled pages can drag down the whole site's quality score in Google's eyes, not just fail to rank themselves
- **Mitigation plan:**
 - Unclaimed/unverified listings: keep minimal, `noindex` them until claimed (still visible in the on-site directory, just not sent to Google)
 - Claimed/verified listings: full SEO treatment (`LocalBusiness` schema, indexed, canonical)
 - and ask the owner for a short unique description at claim time to avoid template duplication

Module 18 – CSC Claim Flow

- "Is this your CSC? Claim it" button on unclaimed listings
- Claim form → CSC ID verification against government registry → owner phone/email OTP
- Admin approval queue (fraud prevention)
- On approval: listing becomes "Verified ", full SEO treatment kicks in

Module 19 – CSC Owner Dashboard + Free Period

- Claimed-owner dashboard: edit photo, hours, services, description
- **6-month free timer** starts at claim-approval date
- Reminder notification as the free period nears its end

Owner-facing UX (confirmed requirements) – two clearly separated entry points:

1. "Claim your CSC listing" – search for their already-listed centre, submit a claim request
2. "Add new CSC centre" – if their centre isn't listed at all, submit a new listing → goes into "pending verification"

Module 20 – CSC Lead Generation

- "Request this service" form on every verified CSC page → lead delivered to owner

(email/SMS/dashboard notification)

- Owner-side lead tracking dashboard

Module 21 – CSC Monetization (post free-period)

- After 6 months: paid plan for continued/enhanced visibility (featured placement, more leads, top-of-list placement)
 - Payment gateway integration (Razorpay/Stripe)
-

Rough Supabase schema sketch (to refine when Module 14 starts)

```
csc_centres
  id, name, address, state, district,
  pincode, lat, lng,
  services_offered (text[]), hours, phone,
  whatsapp,
  status (unclaimed | pending | verified),
  owner_id (nullable, FK -> owners),
  claimed_at, free_period_ends_at,
  description, created_at

owners
  id, name, phone, email, auth_user_id (FK
-> Supabase auth.users),
  created_at
```

```
claims
  id, csc_centre_id (FK), owner_id (FK),
  status (pending | approved | rejected),
  csc_id_proof, submitted_at, reviewed_at,
  reviewed_by

leads
  id, csc_centre_id (FK), visitor_name,
  visitor_phone,
  service_requested, message, created_at,
  status (new | contacted | closed)

comments (Module 15, but same DB)
  id, service_id, name, message,
  created_at, status (visible | flagged)

subscribers (Module 16, but same DB)
  id, email, phone, whatsapp_opted_in,
  service_id (nullable), created_at
```

Open decisions for later (not yet made)

- Exact CSC seed-data source (government dataset vs. manual entry)
- Paid plan pricing/tiers for Module 21

- Whether blog (Module 9) content will be written in-house or need a lightweight CMS later